Week 7

What Is Machine Learning?

Machine Learning (ML) is a **subset of AI** that enables computers to learn patterns from **data** and make **predictions or decisions** without explicit programming.
Examples: predicting house prices, classifying flowers, detecting spam.

---

2 Types of Machine Learning

Machine Learning splits into **three main families**:

1. Supervised Learning

- Learns from **labelled** data
- Input → Output mapping
- Tasks: **Classification**, **Regression**

2. Unsupervised Learning

- Learns from **unlabelled** data
- Finds patterns (clusters, structure)
- Tasks: **Clustering**, **Dimensionality Reduction**

3. Reinforcement Learning

- Agent interacts with environment
- Learns to **maximise a reward**
  (Covered in detail in Week 11)

Supervised Learning

✔️ Classification vs Regression

Classification

- Predict **categories**
    - e.g., species of flower, spam vs not spam
- Example: Iris dataset (Setosa, Versicolor, Virginica)

Regression

- Predict **continuous values**
    - e.g., house price, temperature

Linear Regression (Regression Algorithm)

Used to model the relationship between **input x** and **continuous output y**.

Equation of the line

[
y = mx + b + \varepsilon
]
Where:

- **m** = slope
- **b** = intercept
- **ε** = error term

✔️ How do we find *m* and *b*?

Using the **Least Squares Method**:

- Minimize sum of squared residuals:

$$min \sum (y_1 - (b + mx_1))^2$$

Initial values of m and b are assigned randomly

True values (*label*)

Predicted values

Training data (samples)

-

✔️ Optimisation: Gradient Descent

Steps:

1. Initialise m and b randomly
2. Compute gradients (partial derivatives)
3. Update:
   [
   m = m - \alpha \frac{\partial J}{\partial m}
   ]
   [
   b = b - \alpha \frac{\partial J}{\partial b}
   ]
4. Repeat until converged
5.

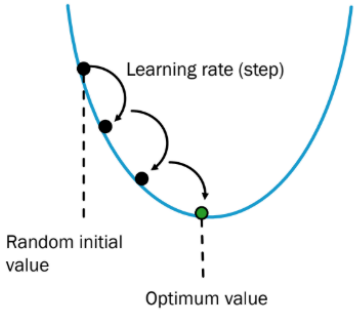
GRADIENT DESCENT (OPTIMISATION)

- Iterative optimisation to find the minimum of a function (cost function/loss function) by adjusting the model parameters in the direction of the steepest descend.
- The idea is to update the parameters in the direction that minimises the cost function.
- Algorithm:

1. Initialise m and b with random values
2. While difference > threshold or there is no change (**convergence**) do:
   1. Compute how much the parameters need to change (by finding the partial derivatives of the cost function with respect to each parameter)
   2. Update parameters m and b using:

$$m = m - (LearningRate * \frac{\partial J}{\partial m})$$

$$b = b - (LearningRate * \frac{\partial J}{\partial b})$$



Where **α = learning rate**.

✔️ Evaluation Metric

Mean Squared Error (MSE):
$$
\text{MSE} = \frac{1}{n} \sum (y - \hat{y})^2
$$

🌳 Decision Trees (Classification Algorithm)

Decision Trees classify data by asking **a sequence of feature-based questions**.

✔️ Key parts:

- **Root node**: first question/feature
- **Splitting criteria**: rule used to split dataset
- **Leaf nodes**: final prediction

✔️ How Decision Trees Work

1. Select the **best attribute**
2. Split data accordingly
3. Repeat recursively
4. Stop when:
    - All samples are same class
    - Max depth reached
    - No improvement

✔️ Attribute Selection Measure

Most commonly used: **Gini Index**
$$
Gini(D) = 1 - (p_1^2 + p_2^2)
$$

- Lower Gini → purer node

4️⃣ Unsupervised Learning

💠 K-Means Clustering

Used to group unlabelled data into **K clusters**.

✔️ Algorithm Steps

1. Choose **K** initial random centroids
2. Assign each point to nearest centroid
3. Recompute centroids (mean of cluster)
4. Repeat until no change / max iterations

✔️ Output

- K clusters
- Centroid for each cluster

✔️ Applications

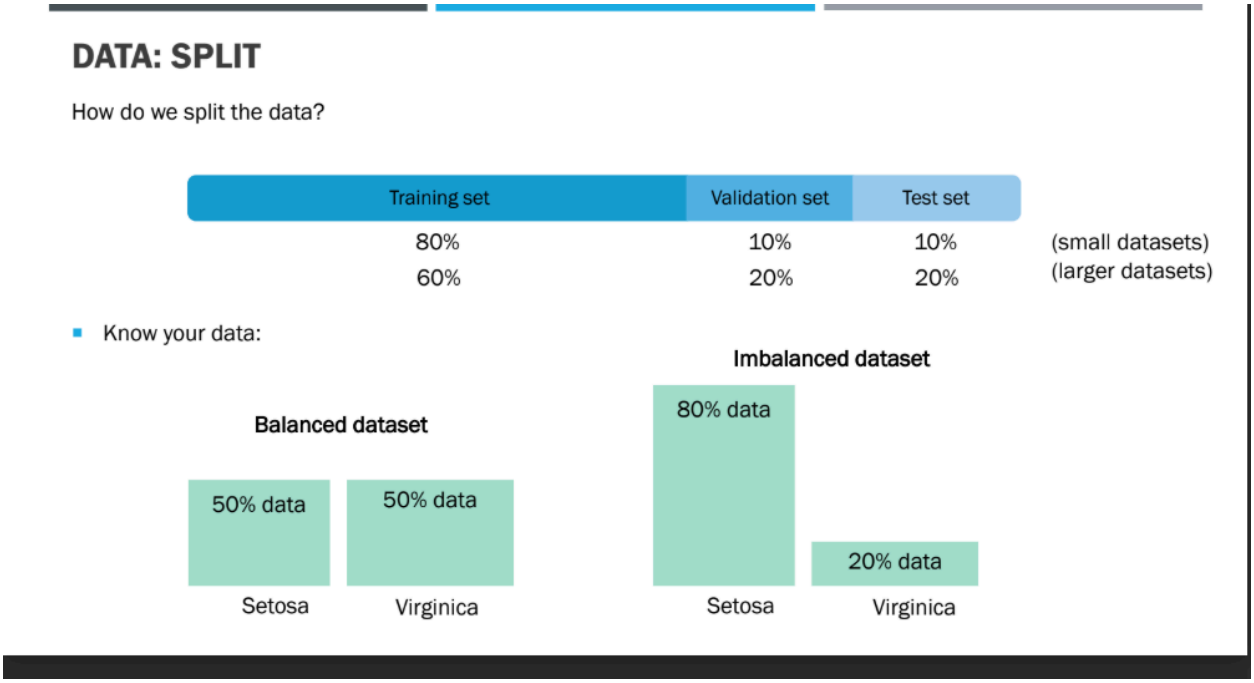
- Customer segmentation
- Anomaly detection
- Pattern discovery
- Image compression

5️⃣ Data in Machine Learning

✔️ Types of Data

- **Numerical** (quantitative): e.g., 5.1, 3.2
- **Categorical**: e.g., “red”, “Setosa”

✔️ Data Splitting



Typical split:

- **Training:** 70–80%
- **Validation:** 10–20%
- **Testing:** 10–20%

Purpose:

- Train → learn patterns
- Validate → tune model
- Test → measure real-world performance

6️⃣ Imbalanced Datasets

Example:

- 95 samples of class A
- 5 samples of class B

A stupid model that always predicts **A** gets **95% accuracy → misleading!**

✔️ Solution: Look at per-class metrics instead.

7️⃣ Confusion Matrix

A table showing performance of a classification model.

	Predicted +	**Predicted –**
Actual +	TP	FN
Actual –	FP	TN

Where:

- **TP:** correctly predicted positives
- **FP:** incorrectly predicted positives
- **FN:** missed positives
- **TN:** correctly predicted negatives

8️⃣ Evaluation Metrics

- **Accuracy** = (TP + TN) / total
- **Precision** = TP / (TP + FP)

- **Recall** = TP / (TP + FN)
- **Specificity** = TN / (TN + FP)
- **F1 Score** = harmonic mean of precision & recall

WEEK 7 SUMMARY ML = data-driven pattern learning

- **Supervised** = labelled → regression + classification
- **Linear Regression:** fit line using least squares + gradient descent
- **Decision Trees:** rule-based classification using splits + Gini Index
- **Unsupervised:** no labels → K-means clustering
- Know your data: numerical vs categorical
- Always split data (train/val/test)
- Accuracy can lie → use confusion matrix and metrics

---

week 8

1. What Are Artificial Neural Networks?

ANNs are computational models inspired by the human brain.
They help computers **learn complex, non-linear relationships** between inputs and outputs.

Why important?
They enable systems like:

- Cancer detection
- Image recognition
- Weather prediction
- Chatbots
- Stock prediction
- Shopping/interest prediction

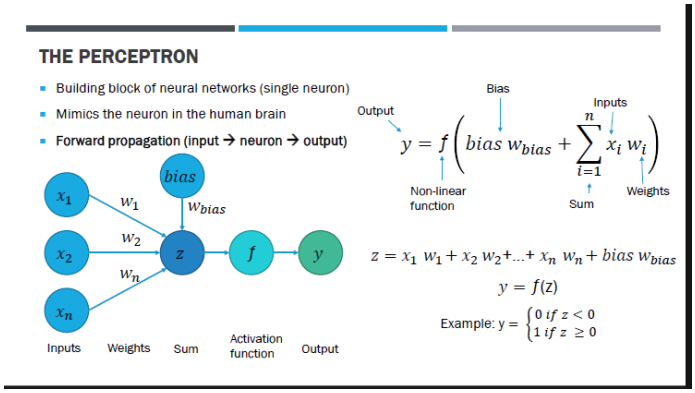
---

2. The Perceptron (Single Neuron)

A perceptron has:

Inputs → Weights → Summation → Activation Function → Output

Formula:



- x_i = inputs
- w_i = weights
- **b** = bias
- **f** = activation (e.g. step function)

✔ Forward Propagation

Forward propagation is how the network **takes inputs and produces an output**.

You multiply inputs by weights → add bias → pass through activation → get output.

✔ Examples covered:

- Cancer risk prediction
- Spam detector
- Logic operations (except XOR)

---

3. How Do We Find the Weights?

We train the perceptron using labelled data.

Steps:

1. Choose random weights
2. Calculate output for each sample
3. Compare prediction vs actual
4. Adjust weights to improve accuracy
5. Repeat until accuracy is acceptable

---

4. Multi-Layer Neural Networks (MLPs)

ANN structure includes:

Input Layer

Receives raw data.

Hidden Layers

Learns patterns (more layers → can learn more complex relationships).

Output Layer

Produces the final prediction.

Each output has its own set of weights for all inputs.

Example:

- Iris dataset → 4 inputs, 1 output
- Images → pixel values as inputs (e.g., 15×12 = 180 inputs)

---

5. Inputs, Outputs & Hidden Layers

Inputs

Come from features in your dataset.

Outputs

Depend on task:

- Binary classification → 1 output (0/1)
- Multi-class → one output per class
- Regression → 1 numerical output

Hidden Layers

- No strict rule
- More complex problems → more layers/neurons
- Easy tasks → fewer layers

---

6. Data Preparation & Splitting

Typical splits:

- **80%** training
- **10%** validation
- **10%** test

OR

- **60%** train
- **20%** validation
- **20%** test

⚠ **Balanced vs imbalanced data matters!**

An imbalanced dataset can show high overall accuracy but fail one class completely.

Example:
90 cats + 10 dogs → predicting everything as “cat” gives **90% accuracy**, but is useless.

---

7. Problems With Imbalanced Data

Accuracy can be misleading.

Always check:

- Accuracy per class
- Confusion matrix
- Possibly use weighted loss functions

---

8. Training a Neural Network

Goal: **Find the best weights that produce the lowest error.**

Training steps:

1. Randomly initialise weights
2. Forward propagation
3. Calculate loss
4. Backpropagation (compute gradients)
5. Update weights (Gradient Descent)
6. Repeat for many epochs

---

9. Loss Functions (For Reference)

- **Binary classification** → Cross-entropy
- **Multiclass classification** → Softmax + Cross-entropy
- **Regression** → Mean Squared Error (MSE)

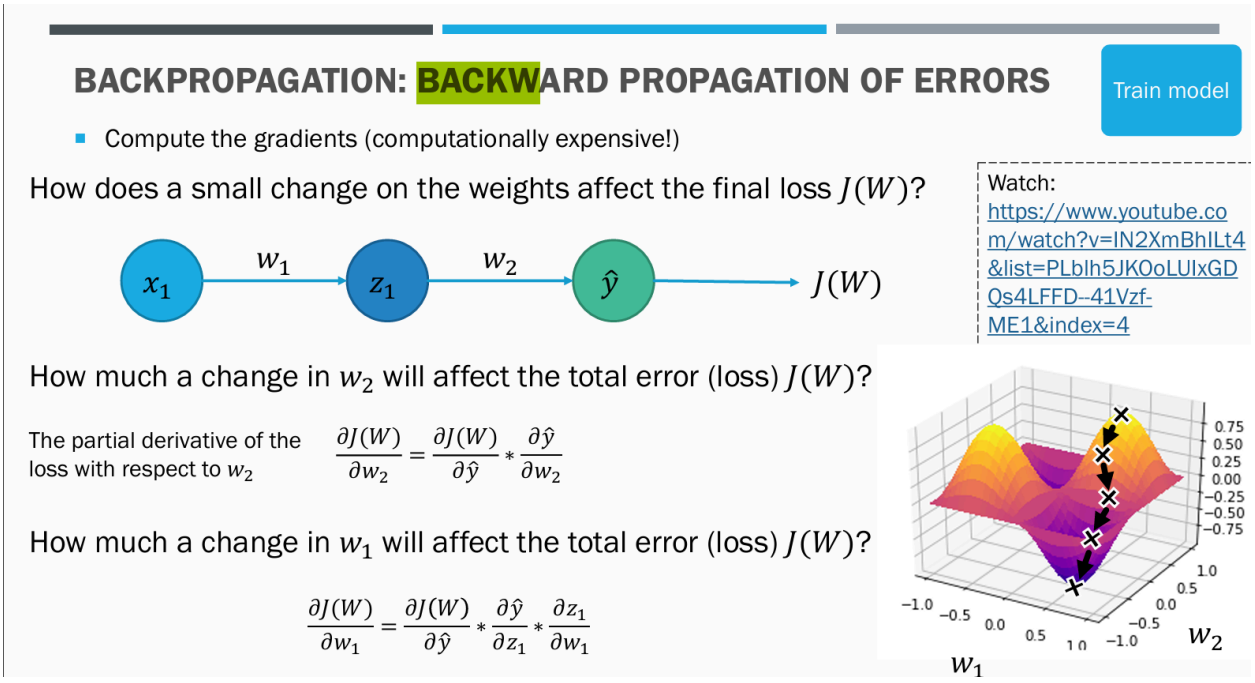
Loss must be **differentiable**.

---

10. Backpropagation

Backpropagation adjusts the weights so that **future predictions become more accurate**.

Backprop computes how each weight contributed to the error:



It adjusts weights to reduce total error.

This is computationally expensive but essential.

---

11. Learning Rate

Controls how big a weight update step is.

- Too large → overshoots, unstable
- Too small → very slow training
- Good learning rate → steady improvement

---

12. Overfitting

Network learns training data too well and performs badly on new data.

Solutions:

- **Dropout** (turn off 50% neurons randomly during training)
- **Early stopping**
- **More data**
- **Regularisation**

---

13. Evaluating the Model

Monitor:

- Training accuracy
- Validation accuracy

Look for:

- Underfitting → low both
- Overfitting → high train, low validation
- Good fit → both rise and stabilise

NOTES

A perceptron can have **as many inputs as you want** — there is **no fixed limit**.

✔️ Key idea:

A perceptron is just a weighted sum of its inputs:

z=w1x1+w2x2+⋯+wnxn+bz = w_1x_1 + w_2x_2 + \dots + w_nx_n + bz=w1x1+w2x2+⋯+wnxn+b

So the number of inputs (**n**) can be:

- 2 inputs
- 10 inputs
- 180 inputs (e.g., image pixels)
- 30,000 inputs (text features)
- Even **millions** (in large-scale models)

✔️ What determines the number of inputs?

Your dataset.

- If your input has 4 features → perceptron has 4 inputs
- If input is a 100×100 image → perceptron has 10,000 inputs
- If input is a 784-pixel MNIST image → 784 inputs

Each **input has exactly ONE weight** in a perceptron.

✔️ Meaning:

If a perceptron has **n inputs**, then it has:

- **n weights** (one for each input)
- **+ 1 bias weight**

Absolutely — here are **clear, exam-style Q&A** based on perceptrons, neural networks, forward/backward propagation, activation functions, and ANN basics.
These are perfect for revision, assignments, or practice! 🔥

TOPIC: Perceptrons (Questions & Answers)

Q1: How many inputs can a perceptron have?

A: Any number — it depends on the number of features in the dataset. There is no fixed limit.

Q2: How many weights does a perceptron have?

A: One weight per input, plus a bias weight.

Q3: What is the role of the bias in a perceptron?

A: It shifts the activation function left or right and helps the perceptron learn patterns that don't pass through the origin.

Q4: Can a single perceptron solve the XOR problem?

A: No. XOR is not linearly separable, so it requires at least one hidden layer.

Q5: Write the perceptron output formula.

A:
[
y = f\left(\sum_{i=1}^n w_i x_i + b\right)
]

🌟 TOPIC: Forward Propagation (Questions & Answers)

Q6: What is forward propagation?

A: The process where inputs pass through the network layer by layer to produce a prediction.

Q7: What is the purpose of the activation function during forward propagation?

A: To introduce non-linearity so the network can learn complex patterns.

Q8: How do you compute the output of a neuron?

A: Multiply inputs by weights → add bias → apply activation function.

Q9: Why do we need activation functions like ReLU or Sigmoid?

A: Without them, the neural network would behave like a linear model and could not learn non-linear relationships.

🌟 TOPIC: Backpropagation (Questions & Answers)

Q10: What is backpropagation?

A: The process of sending the error backwards through the network to adjust the weights.

Q11: Why do we need the gradient in backpropagation?

A: The gradient tells us the direction to change each weight to reduce error.

Q12: What is the learning rate?

A: A hyperparameter that controls how big the weight updates are during backpropagation.

Q13: What happens if the learning rate is too high?

A: The model becomes unstable — it may overshoot minimum loss and fail to learn.

Q14: What happens if the learning rate is too low?

A: Training becomes very slow and may get stuck.

Q15: What is the loss function?

A: A measure of the error between the predicted output and the actual target.

🌟 TOPIC: Neural Network Basics (Questions & Answers)

Q16: What is a hidden layer?

A: A layer between the input and output layers where feature extraction happens.

Q17: Why do we split data into training, validation, and testing sets?

A: To train the model, tune hyperparameters, and evaluate performance on unseen data.

Q18: What is overfitting?

A: When a model learns the training data too well and performs poorly on new data.

Q19: How do we reduce overfitting in neural networks?

A: Use dropout, early stopping, regularisation, or collect more data.

Q20: What are common activation functions?

A: Sigmoid, ReLU, Tanh, Softmax.

Q21: What is the softmax function used for?

A: Multi-class classification; converts outputs into probabilities that add up to 1.

Q22: What is the role of weights in a neural network?

A: They determine how strongly each input influences the neuron's output.

🌟 TOPIC: Backprop Example (Q&A Style)

Q23: During backprop, which weights get updated the most?

A: Weights that contributed most to the error (largest gradients).

Q24: When do we perform backpropagation?

A: Immediately after forward propagation and loss calculation.

Q25: What rule is used to compute gradients in backpropagation?

A: The chain rule from calculus.

week 9

1. Why Computer Vision?

- Computer vision enables machines to interpret visual information like images and videos.
- Applications:
    - **Face recognition**
    - **Medical imaging** (detecting haemorrhages in retina)
    - **Robotics**
    - **Self-driving cars**
    - **Surveillance**
- Object detection is crucial because perception is a major bottleneck in autonomy.

---

2. What is Object Detection?

Object understanding involves several tasks:

Task	Description
Classification	What is in the image?
Localization	Where is the object? (bounding box)
Detection	Find & classify multiple objects
Segmentation	Pixel-level classification

Difficulty increases from classification → segmentation.

---

3. What Computers See (Traditional Vision)

Before deep learning, features were **hand-crafted**.

Hand-crafted Features

- **Low-level features**: edges, corners.
- Techniques:
    1. **SIFT**
    2. **SURF**
    3. **HOG**
    4. **LBP**
    5. **Color histograms**
- Recognition pipeline:
    1. Extract features
    2. Feed into classifier (e.g., SVM)

📌 **Limitations:**

- Features are *not learned*.
- Hard to hand-design optimal features.
- Sensitive to changes (lighting, orientation, noise).
- Struggles with intraclass/interclass variation.

---

4. Motivation for Deep Learning

Instead of hand-designing features → **learn features automatically**.

Main ideas:

- Learn **feature hierarchy** directly from pixels.
- Lower layers learn edges → mid layers learn shapes → high layers learn full objects.
- Deep learning fills the “representation gap”.

---

5. Learning Feature Hierarchies

- Deep networks use **multiple layers**, each extracting progressively more abstract features.

Example hierarchy:

Layer 1 → Edges
Layer 2 → Object parts (corners, textures, patterns)
Layer 3 → Whole objects

Train all layers jointly using **backpropagation**.

---

6. Supervised vs Unsupervised Learning

Supervised

- End-to-end learning using labelled datasets.
- Works well with large labelled data.

- Model architecture crucial (CNNs, etc.).

Unsupervised

- Learns statistical structure from unlabelled data.
- Layer-wise training (historically used when labels were limited).

---

7. Convolution & Feature Detection

CNNs detect features using **learned filters**.

Convolution Operation

- Slide a filter over the image.
- Compute weighted sum → generate **feature map**.

Why convolution?

- Exploits spatial structure.
- Local connectivity → fewer parameters than fully connected layers.
- Filters are learned via backpropagation.

Edge detection

Filters like Sobel, Prewitt detect vertical/horizontal edges.

---

8. Convolutional Neural Networks (CNNs)

Core components:

a) Convolutional Layer

- Uses filters (kernels) to detect patterns.
- Output volume defined by:
    - **Height, Width** (depends on stride/padding)
    - **Depth** = number of filters

b) Activation Function (ReLU)

- ReLU: `max(0, x)`
- Removes negative values → non-linear.
- Avoids vanishing gradient (faster training).

c) Pooling Layer

- Downsampling:
    - Max pooling (most common)
    - Average pooling
- Reduces computation + enforces spatial invariance.

d) Fully Connected Layer

- Uses high-level features for classification.
- Outputs class probabilities (softmax).

---

9. CNN Pipeline Summary

Input Image
→ CONV (feature detection)
→ ReLU (nonlinearity)
→ POOL (downsampling)
→ Repeat (multiple layers)
→ Flatten
→ Fully Connected Layer
→ Softmax (class probabilities)

---

10. CNN Example: Car Classification

- Filters detect features such as:
    - Wheels
    - Windows
    - Headlights
- Final model classifies the image as a car/non-car.

---

11. CNN Architectures & History

History Milestones

- 1980: *Neocognitron* (Fukushima)
- 1989–1998: LeCun et al. → handwritten digit recognition
- 2012: **AlexNet** (Krizhevsky, Sutskever, Hinton)
    - Major ImageNet breakthrough → popularized deep learning

Architectural Concepts

- Using **stacked 3×3 convolutions** instead of single large ones → fewer parameters.
- **1×1 convolutions** for:
    - Dimension reduction
    - Increasing depth efficiently (used in GoogLeNet, ResNet)

---

12. Deep Networks – Tricks of the Trade

Activation functions

Sigmoid

- Outputs 0 to 1
- Slow learning due to saturation
- Not zero-centered

tanh

- Outputs -1 to 1
- Zero-centered but still saturates

ReLU (most used)

- Fast convergence
- Doesn’t saturate in positive region

Weight Initialization

- Depends on network depth & activation function.
- Proper initialization helps avoid:
    - Vanishing gradients
    - Exploding gradients

Regularization

To prevent overfitting:

- **Dropout**
- **Data augmentation**
- **Weight decay**

Training Strategies

- Mini-batches
- Many epochs
- Stochastic Gradient Descent (SGD)
- Backpropagation for updating parameters

---

13. Key Takeaways

- Features are essential for recognition.
- Hand-crafted features → replaced by learned features.
- CNNs automatically learn hierarchical feature representations.
- Key components: Convolution, ReLU, Pooling, Fully Connected Layers.
- Deep networks require careful training (initialization, regularization).
- Modern architectures (AlexNet, VGG, GoogLeNet, ResNet) rely on deep stacks of conv layers.

Below are **clean, structured notes AND a full set of practice questions** based on your PDF tutorial (Week 10 – CNNs), written in the same detailed style as the lecture notes I provided earlier.

📘 Notes: Deep Learning & CNNs (Tutorial Week 10)

(Clear definitions + worked explanations aligned with the tutorial aims)

---

1. Core Concepts & Definitions

Deep Learning

- A subfield of machine learning that uses deep neural networks with many layers.
- Learns hierarchical features directly from raw data (e.g., pixels) without manual feature engineering.

Classification

- Task of assigning an input to one of several predefined categories.
  Example: classifying images as *cat/dog/car*.

Features

- Patterns or attributes extracted from data.
  In images, features may include edges, textures, shapes.

Filters (Kernels)

- Small matrices that slide over the image to detect specific features (e.g., edges, corners).

- Learnable parameters in a CNN.

Convolution

- A mathematical operation where a filter is moved across the image, computing the weighted sum between filter values & image pixels.

- Produces a **feature map** highlighting where features occur.

CNN (Convolutional Neural Network)

- A type of neural network designed for images.

- Automatically learns spatial features via convolution layers.

Convolution Layer

- Applies filters to the input to extract local patterns.

- Output volume depends on:

    - filter size

    - stride

    - padding
    - number of filters (depth)

Pooling Layer

- Downsamples the feature map to reduce size & retain important features.
- Types:
    - **Max pooling:** keep largest value
    - **Average pooling:** compute mean

ReLU (Rectified Linear Unit)

- Activation function: `ReLU(x) = max(0, x)`
- Adds nonlinearity & prevents vanishing gradients.

Computer Vision

- Field where computers interpret visual information.
- Used in object detection, face recognition, medical imaging, robotics, etc.

---

2. Convolution: How to Calculate

Given:

- 3×3 kernel
- Stride = 1

Steps:

1. Align the 3×3 kernel with the top-left corner of the image.
2. Multiply corresponding values.
3. Add them up → this is one output pixel.
4. Slide the filter by 1 pixel (stride=1).
5. Repeat until full feature map is produced.

Formula:

Output = Σ (Image_pixel × Kernel_value)

The output feature map dimensions:

Output_height = (H - K)/S + 1

Output_width = (W - K)/S + 1

Where:
H,W = input height/width
K = kernel size
S = stride

---

3. Pooling

Given a feature map, pooling reduces the spatial size.

a) Max Pooling (2×2, stride 2)

- Split input into 2×2 blocks
- Take **maximum** value from each block

b) Average Pooling (2×2, stride 2)

- Split into 2×2 blocks
- Compute **mean** of each block

Pooling:

- Reduces computation
- Gives translation invariance
- Prevents overfitting by reducing parameter count

---

4. Training a CNN (From Tutorial Notebook)

Typical steps in a Jupyter notebook:

Building the Model

- Define layers:
    - Conv → ReLU → Pool → (repeat)
    - Dense layers → Softmax

Compiling

- Choose:
    - Loss function (e.g., categorical crossentropy)
    - Optimizer (SGD, Adam)
    - Metrics (accuracy)

Training

- Use `fit()` with training data
- Observe:
    - Loss decreases
    - Accuracy increases

Evaluating

- Test on unseen data
- Compare training vs validation accuracy
- Check for underfitting or overfitting

Improving Performance

- Add layers
- Increase filters
- Use dropout
- Use data augmentation
- Tune learning rate
- Early stopping

Section B — Convolution Calculations

2. Compute the convolution

Given a 3×3 kernel and stride=1, calculate the full output feature map.

(Add your own numbers from the PDF diagram.)

Steps to answer:

- Multiply kernel with each 3×3 region
- Add results
- Slide by 1 pixel
- Continue until full matrix is calculated

Section C — Pooling Calculations

3. Given the feature map below, compute:

a) 2×2 Max pooling (stride 2)
b) 2×2 Average pooling (stride 2)

(Use values in the PDF whenever answering.)

Max pooling example block:

[3 5]

[2 1]

Max = 5

Average pooling example block:

(3 + 5 + 2 + 1) / 4 = 2.75

Section D — Conceptual Questions

4. Explain why pooling layers are used in CNNs.

Expected answer:

- Reduce computation
- Reduce overfitting
- Provide spatial invariance
- Capture most important features

---

5. Why is ReLU preferred over sigmoid in deep networks?

Expected answer:

- Avoids vanishing gradients
- Faster training
- Simpler computation
- Works well in practice

---

6. Describe how a CNN learns filters.

Expected answer:

- Filters start with random values
- Forward pass → compute predictions

- Backpropagation → compute gradients
- Filters update to reduce loss

---

7. Why is convolution better than fully connected networks for images?

Expected answer:

- Preserves spatial structure
- Requires fewer parameters
- More efficient
- Captures local patterns

Section E — Practical / Notebook-Based

8. During training your CNN reaches 99% training accuracy but 72% validation accuracy. What is happening? What should you do?

Expected answer:

- Overfitting
- Use dropout, data augmentation, regularization
- Reduce model complexity
- Use early stopping

---

9. What does an epoch mean in training? What are mini-batches?

---

10. How does data augmentation improve CNN performance?

Section F — Advanced Thinking

11. Explain how feature hierarchies emerge in CNNs.

Expected structure:

- Lower layers: edges
- Middle layers: textures, shapes
- Higher layers: full objects
- Learned through joint training via backpropagation

---

12. What effect does increasing stride have on convolution output?

Expected explanation:

- Fewer output pixels
- More aggressive downsampling
- Faster computation but less detail

---

13. Compare max pooling vs average pooling. When is each preferred?